RSpec Execution Regression Post-Mortem

Date: 2026-04-24 Status: Resolved Scope: RSpec integration, mutation child execution, baseline coverage bootstrap, smoke coverage

Summary

Henitai regressed in its own RSpec execution path and became unable to run mutation testing reliably on the Henitai repository itself. The visible symptoms were:

- mutant children timing out
- child logs showing No examples found. or stopping after runner_run_start
- baseline coverage bootstrap failing intermittently on repo-scale runs
- real zero-example runs being misclassified

The regression had two related technical causes:

1. The mutant child path had drifted away from the documented execution model. Mutation activation is process-local in Henitai, so the activated mutant and the test run must stay in the same forked child process. Earlier work had introduced an additional execution boundary and weakened that contract.
2. The RSpec boot path relied on RSpec::Core::Runner.run(...) and related default discovery behavior in places where Henitai already had an explicit file list. On the Henitai repository this triggered expensive repo-scale discovery through Configuration#get_files_to_run, which was cheap enough on tiny smoke fixtures to go unnoticed but pathological on the real codebase.

The failure was not caused by mutation activation itself. It was caused by an invalid execution workflow around RSpec boot and discovery.

Impact

- bundle exec henitai run on the Henitai repository became unreliable or unusable.
- Mutation runs produced timeouts instead of actionable statuses.
- Diagnostic logs were initially too weak to separate real timeouts, discovery failures, and misclassification.
- Confidence in the basic RSpec integration dropped because toy smoke projects stayed green while the real repo failed.

User-Visible Symptoms

- Narrow dogfood runs like bundle exec henitai run ’Henitai::Integration’ produced many T timeout results.
- Some mutant logs contained No examples found. even though the selected spec files existed and contained examples.
- Baseline coverage bootstrap failed with Henitai::CoverageError on the repo.
- The existing smoke suite reported:
  - smoke:rspec ok
  - smoke:minitest ok even while the real repository run was broken.

Detection

The issue was detected by dogfooding Henitai against its own repository, not by CI or by the existing integration smoke suite.

That mattered. The existing smoke fixtures were real projects, but they were too small to trigger the discovery cost and state problems that

appeared on the Henitai repository.

Timeline

1. Initial symptom characterization

- Reproduced the failure with narrow subjects and --jobs 1 --all-logs.
- Observed repeated timeouts and empty or unhelpful logs.
- Confirmed that No examples found. was not legitimate for the selected repo spec files.

2. Child-process instrumentation

- Added gated child diagnostics under HENITAI_DEBUG_CHILD=1.
- Logged selected files, loaded-feature state, RSpec world example counts, and runner lifecycle markers.
- Used timeout-triggered child thread dumps to identify where the child was actually stalling.

3. Root-cause isolation

- Confirmed that the execution problem was in RSpec boot and discovery, not in mutant activation.
- Reproduced the same stall in a direct subprocess outside the full mutation pipeline.
- Verified from thread dumps that RSpec::Core::Configuration#get_files_to_run and its directory scanning path were involved.

4. Architecture correction

- Restored the same-process mutant child contract for activated mutant execution.
- Reworked the baseline suite runner to use an explicit file list instead of plain Runner.run(...).
- Tightened classification for zero-example child runs.

5. Smoke and test hardening

- Added a repo-level dogfood RSpec smoke path.
- Updated integration specs to test the real current runner shape instead of the old Runner.run(...) assumptions.
- Fixed Steep surface declarations for the new debug helper methods.

Root Cause

Root Cause 1: Execution contract drift

Henitai mutates code by injecting an activated method definition into a live Ruby process. That means mutation activation is process-local by design. The mutant must be activated and the tests must execute in the same forked child.

The RSpec integration had drifted away from that contract. Once that happened, debugging the later symptoms became harder because:

- some failures looked like timeouts
- some looked like No examples found.
- some were misclassified

This was an architecture regression, not just a failing test.

Root Cause 2: Default RSpec discovery on a repo-scale codebase

Henitai already knew which spec files it wanted to run. Despite that, parts of the RSpec path still used default runner behavior that lets RSpec discover files on its own.

On the tiny smoke fixtures this was cheap enough to pass. On the Henitai repository it triggered the slower discovery path and caused the observed stalls.

The correct model for Henitai is:

1. decide the exact file list in Henitai
2. seed RSpec with that file list explicitly
3. load only those files
4. run those example groups

Not:

1. hand control back to Runner.run(...)
2. let RSpec rediscover the suite from the current working directory

Contributing Factors

1. Smoke suite too small

The existing RSpec and Minitest smoke fixtures were real projects, but they were toy-sized. They were good at checking that the end-to-end pipeline was not completely broken. They were not good at exposing repo-scale discovery cost or child-runner state problems.

2. Missing contract tests

Some integration specs mocked or stubbed the old runner boundary and therefore did not protect the actual invariant:

- mutation activation and test execution must share the same forked child
- zero-example child runs must not degrade into :survived or :timeout

3. Weak initial diagnostics

The original logs did not expose enough of the child lifecycle to distinguish:

- hang in discovery
- hang in setup
- zero-example result
- compile error
- true timeout

4. Test pollution in the spec suite

At least one updated RSpec unit example still invoked the real run_tests path, which leaked behavior into later examples and caused unrelated failures in integration specs for Minitest. That was fixed, but it showed that the spec boundary itself had become too loose.

Why The Existing Smoke Tests Worked

They worked because they exercised the same code on much smaller projects.

That difference was enough to hide the bug:

- small fixture repos do not stress repo-scale file discovery

- their spec trees are tiny
- the cost of accidental default RSpec discovery stays low
- some child-runner state issues do not surface when the suite is
  trivial

So the smoke suite was not fake. It was simply not representative enough
for this failure mode.

Resolution

The fix set had four parts.

1. Restore the correct execution model

- keep mutation activation and RSpec execution in the same forked child
- remove reliance on the invalid extra execution boundary

2. Bypass default discovery when Henitai already knows the file list

- build and configure the RSpec runner directly
- set files_to_run explicitly
- call load_spec_files
- execute run_specs on the selected example groups

3. Classify zero-example failures conservatively

Zero-example child runs are now treated as :compile_error in the
relevant failure shapes instead of being allowed to drift into a
misleading status.

4. Strengthen regression coverage

- added repo-level dogfood smoke coverage for the baseline RSpec path
- updated integration specs to match the real runner structure
- updated type signatures for the added debug helper surface

What Changed

The principal corrective changes landed in:

- lib/henitai/integration.rb
- lib/henitai/scenario_execution_result.rb
- rakelib/integration_smoke.rake
- spec/henitai/integration/rspec_spec.rb
- spec/infra/integration_smoke_projects_spec.rb
- sig/henitai.rbs

Verification

The fix was verified with:

- focused RSpec regressions for:
  - zero-example child classification
  - runner lifecycle logging
  - file-list setup before spec loading
- repo-level baseline smoke through smoke:dogfood_rspec
- bundle exec rake smoke:integration:all
- bundle exec steep check
- focused affected-area spec runs
- renewed mutation runs on the Henitai repository

At the end of the work:

- the smoke suite passed

- Steep passed
- affected integration and classifier specs passed
- mutation runs on Henitai were running again

Lessons Learned

1. The architecture contract must be executable

If a design rule is essential, it needs a contract test. In this case:

- same-process mutant activation and test execution
- explicit file-list execution for RSpec

2. A tiny smoke suite is not enough

Small real fixtures are still useful, but they are not a substitute for one fast repo-level dogfood smoke that exercises the real execution path.

3. Child diagnostics should be built in early

Mutation frameworks need strong child-process diagnostics because the failure surface is wide:

- fork boundaries
- stdout/stderr capture
- framework boot
- suite discovery
- test hooks
- timeouts

4. Misclassification is almost as bad as crashing

A mutation-testing tool must not silently convert a runner failure into a plausible but wrong mutant status. Wrong feedback is worse than loud failure.

Preventive Actions

Done

- Added a repo-level dogfood RSpec smoke path.
- Hardened zero-example classification.
- Updated integration tests to follow the real runner shape.
- Added debug lifecycle markers and timeout thread dumps.
- Declared the new debug surface in RBS.

Recommended Next

- Add a small repo-level mutant-child smoke, not only a baseline-suite smoke.
- Keep the dogfood smoke in CI as a required gate.
- Add an explicit regression around the ?same process? mutation activation contract.
- Treat unexpected zero-example runs as a named runner/discovery failure mode in reporting, not only as a conservative compile-error fallback.
- Clean up duplicated ScenarioLogSupport responsibilities in lib/henitai/integration.rb to make the integration easier to reason about.

Conclusion

This incident was not a random flake. It exposed a real architectural problem: Henitai had let RSpec execution drift away from the core

constraints of its own mutation model, and the existing smoke coverage was too small to catch it.

The repair was successful because it focused on the root cause:

- restore the execution contract
- stop delegating file selection back to default RSpec discovery
- verify the real path on the real repository

That is the standard the integration needs going forward.